# Хранение данных. Volumes и Bind Mounts

Постоянные данные Docker: Nginx, Alpine и PostgreSQL 18

# Цели занятия

- Объяснять, почему файлы в записываемом слое теряются после удаления контейнера.
- Выбирать подходящий способ хранения: Docker Volume или Bind Mount.
- Создавать, подключать и диагностировать именованные Docker Volumes.
- Запускать PostgreSQL 18 с правильно подключённым постоянным томом.
- Проверять сохранность SQL-записей после удаления и пересоздания контейнера.

# На что опираемся

- Лекция 1: Docker Image и Docker Container — разные объекты.
- Лекция 2: docker run, stop, rm, exec, logs и inspect.
- Лекция 3: получение и исследование Docker-образов.
- Лекция 4: создание собственного Docker-образа с помощью Dockerfile.
- В этой лекции не требуется Express.js: SQL-команды выполним внутри контейнера PostgreSQL.

# Учебные вопросы

| № | Учебный вопрос |
|---|---|
| 1 | Файловая система контейнера и проблема сохранности данных |
| 2 | Docker Volumes: создание, подключение, управление |
| 3 | Bind Mounts: связь с каталогами хоста |
| 4 | PostgreSQL с постоянным хранилищем |
| 5 | Проверка сохранности, диагностика и безопасность |

# Почему данные контейнера могут исчезнуть

Записываемый слой контейнера и его жизненный цикл

# Контейнер: неизменяемые слои и запись

- Образ формируется из read-only слоёв.
- При создании контейнера Docker предоставляет отдельный записываемый слой.
- Изменения внутри этого слоя принадлежат конкретному контейнеру.
- При удалении контейнера его записываемый слой удаляется.

**Главная идея**

**Остановка контейнера не равна удалению, но удаление контейнера удаляет его записываемый слой.**

# Почему это важно для программиста

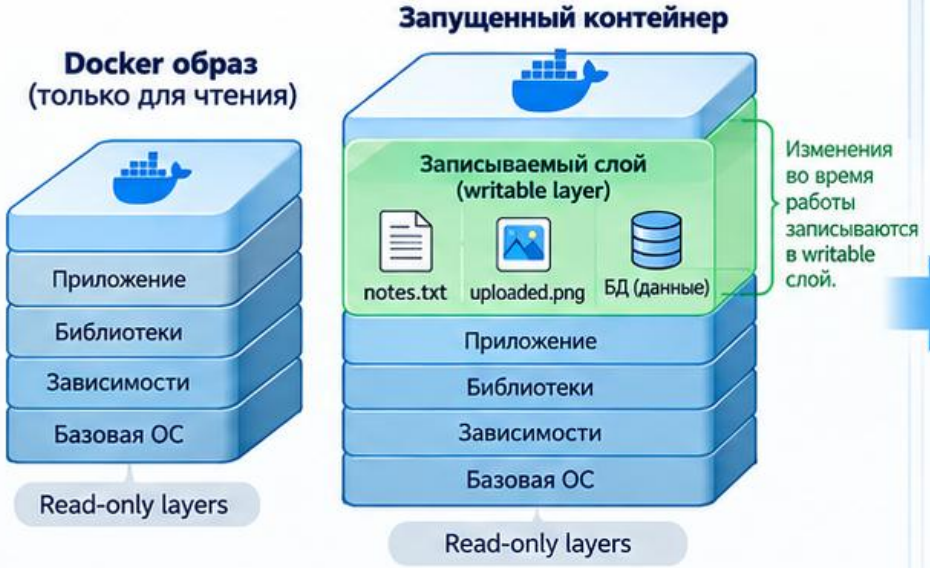
- Сайт на HTML, CSS и JavaScript может использовать смонтированный каталог с файлами.
- Python-приложение может записывать заметки и файлы в подключённый том.
- PostgreSQL постоянно изменяет файлы таблиц и журналов.
- При обновлении приложения контейнер часто пересоздают — данные должны оставаться.

# Записываемый слой контейнера и потеря данных

Почему данные внутри контейнера могут исчезнуть после его удаления

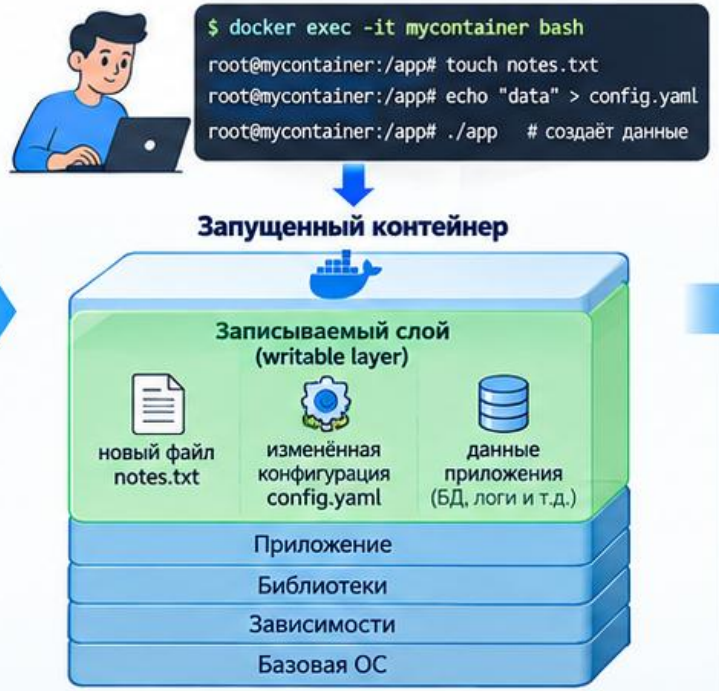
## 1 Образ и запущенный контейнер

Контейнер создаётся из образа и имеет дополнительный записываемый слой сверху.




## 2 Работа с контейнером

Пользователь работает с запущенным контейнером и создаёт в нём данные.




## 3 Удаление контейнера

При удалении контейнера его записываемый слой удаляется вместе с ним, и все данные в нём теряются.

```
$ docker rm mycontainer
```




**Данные в записываемом слое не являются постоянными!**

После удаления контейнера все файлы, изменения конфигурации и данные приложетеия теряются.

## Без volume / bind mount




- данные записываются в writable layer
- после удаления контейнера данные теряются
- подходит только для временных данных

## С volume или bind mount




- данные хранятся вне контейнера
- пересоздание контейнера не удаляет данные
- подходит для баз данных и пользовательских файлов

## Что важно понять

1. Контейнер создаётся из образа.
2. Образ остаётся, даже если контейнер удалён.
3. Записываемый слой контейнера временный.
4. Для сохранения данных нужны volumes или bind mounts.

# Что происходит без постоянного хранилища

1. **Шаг 1** — Создаём контейнер из образа.
2. **Шаг 2** — Приложение записывает данные внутрь записываемого слоя.
3. **Шаг 3** — Контейнер останавливается: данные пока остаются.
4. **Шаг 4** — Контейнер удаляется: его записываемый слой исчезает.
5. **Шаг 5** — Новый контейнер из того же образа не наследует эти изменения.

# Остановить или удалить контейнер

### docker stop

- Основной процесс останавливается.
- Контейнер остаётся в docker ps -a.
- Его записываемый слой остаётся.

### docker rm

- Объект контейнера удаляется.
- Его записываемый слой удаляется.
- Именованный Docker Volume сохраняется, пока его не удалят отдельно.

# Docker Volumes

Постоянное хранилище, которым управляет Docker

# Определение Docker Volume

- Volume — управляемая Docker область хранения данных.
- Том существует независимо от жизненного цикла конкретного контейнера.
- Один и тот же именованный том можно повторно подключать к новым контейнерам.
- Docker предоставляет команды для создания, просмотра и удаления томов.

# Docker Volume и жизненный цикл контейнера

Как данные сохраняются независимо от создания, остановки, удаления и пересоздания контейнера

### 1 docker run



**Контейнер создан и запущен**

Контейнер подключается к существующему volume

### 2 Работа приложения



**Приложение записывает данные**

Данные сохраняются во volume

### 3 docker stop



**Контейнер остановлен**

Данные по-прежнему хранятся во volume

### 4 docker start



**Тот же контейнер снова запущен**

Использует тот же volume с теми же данными

### 5 docker rm



**Контейнер удалён**

Данные в volume сохраняются

### 6 docker run (новый контейнер)



**Новый контейнер снова подключается к тому же volume**

Данные доступны в новом контейнере




## Главная идея:

**Контейнеры могут создаваться, останавливаться, удаляться и пересоздаваться.**
**Docker Volume продолжает существовать и хранить данные.**

## Без volume



Данные в writable layer контейнера

- Данные находятся в writable layer контейнера.
- После удаления контейнера данные **теряются**.
- Подходит только для временных данных.

## С volume



Данные хранятся в отдельном volume (вне контейнера)

- Данные хранятся вне контейнера.
- После удаления контейнера данные **сохраняются**.
- Подходит для баз данных и пользовательских файлов.

## Практический пример

Команда запуска:

```
docker run -d --name pg1 \
  -v pgdata:/var/lib/postgresql/data \
  postgres:17
```

1. Создаём записи в БД (например, таблицы и данные).
2. Удаляем контейнер: `docker rm pg1`.
3. Создаём новый контейнер с тем же volume.
4. Данные остались — можно работать дальше.

## Что важно понять

1. Volume живёт дольше контейнера.
2. Удаление контейнера не удаляет volume автоматически.
3. Один и тот же volume можно снова подключить к новому контейнеру.
4. Для удаления volume нужна отдельная команда.

# Команды работы с томами

| Команда | Назначение |
| --- | --- |
| docker volume create mydata | Создать именованный том |
| docker volume ls | Просмотреть тома |
| docker volume inspect mydata | Исследовать метаданные |
| docker volume rm mydata | Удалить неиспользуемый том |
| docker volume prune | Удалить неиспользуемые тома: только с осознанной проверкой |

# Создание тома

```bash
sudo docker volume create pgdata-lab
```

**Что важно**

- Команда создаёт именованный том pgdata-lab.
- Повторный вызов с тем же именем не создаёт новый том.
- Проверка: docker volume ls показывает pgdata-lab.

# Просмотр томов и метаданных

BASH

```
sudo docker volume ls
sudo docker volume inspect pgdata-lab
```

### Что важно

- inspect покажет Driver и Mountpoint.
- Фактическое расположение зависит от Docker Engine и конфигурации хоста.
- Не изменяйте файлы тома вручную в каталоге Docker.

# Монтирование тома через --mount

```bash
sudo docker run --rm \
  --mount type=volume,src=pgdata-lab,dst=/data \
  alpine:3.22 \
  sh -c "echo persistent > /data/check.txt"
```

**Что важно**

- src — имя тома Docker; dst — путь внутри контейнера.
- --rm удалит временный контейнер после выполнения команды.
- Именованный том pgdata-lab остаётся после удаления контейнера.

# Читаем файл из нового контейнера

BASH

```
sudo docker run --rm --mount \
  type=volume,src=pgdata-lab,dst=/data \
  alpine:3.22 cat /data/check.txt
```

**Что важно**

- Выполняется новый контейнер из того же образа.
- Ожидаемый результат: persistent.
- Это доказывает, что файл сохраняется вне жизненного цикла первого контейнера.

# Именованный и анонимный том

### Named Volume

- Явное имя: pgdata-lab.
- Проще повторно подключить к другому контейнеру.
- Подходит для файлов базы данных и состояния приложений.

### Anonymous Volume

- Не имеет удобного пользовательского имени.
- Может быть создан автоматически или явно.
- Сложнее отслеживать и повторно использовать.

# Bind Mounts

Монтирование файла или каталога хоста в контейнер

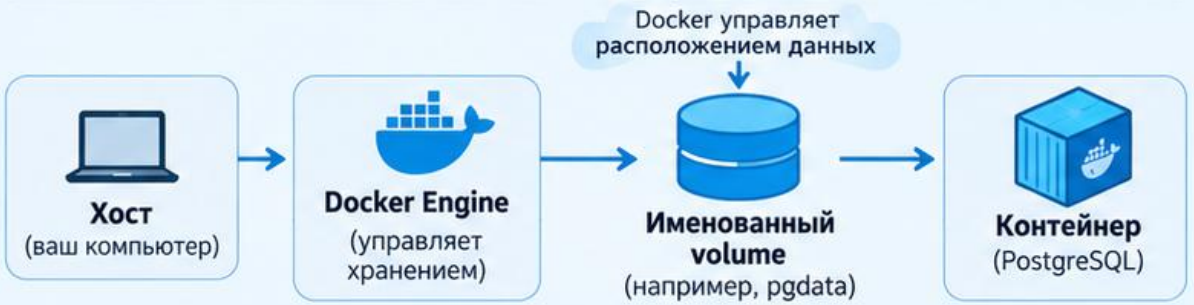
# Определение Bind Mount

- Bind Mount — прямое подключение файла или каталога хостовой системы к пути внутри контейнера.
- В отличие от Volume, источник задаётся конкретным путём хоста.
- Изменение файлов может быть видно как на хосте, так и в контейнере.
- Полезно для разработки и передачи статических файлов или конфигурации.

# Volume и Bind Mount — два способа хранения

Сравнение двух способов подключения данных в Docker

## Docker Volume




- Данные хранятся в области, управляемой Docker
- Удобно для баз данных и постоянных данных
- Независим от структуры каталогов хоста
- Легко переносить и повторно подключать

**Пример: PostgreSQL с постоянным volume**

```
docker run -d --name pg1 \
  -v pgdata:/var/lib/postgresql/data \
  postgres:17
```

## Bind Mount




- Подключает конкретную папку или файл хоста
- Удобно для исходного кода, конфигов и статических файлов
- Изменения на хосте сразу видны в контейнере
- Зависит от структуры каталогов и прав на хосте

**Пример: сайт из локальной папки**

```
docker run -d --name web1 \
  -v /home/user/site:/usr/share/nginx/html \
  nginx:stable
```

## Краткое сравнение

| Критерий | Docker Volume | Bind Mount |
|---|---|---|
| **Кто управляет хранением** | Docker | Хостовая ОС / пользователь |
| **Типичный сценарий** | Базы данных, постоянные данные | Код проекта, конфиги, статические файлы |
| **Переносимость** | Выше | Ниже |
| **Прямая работа с файлами хоста** | Ограничена | Да |
| **Удобство в разработке** | Среднее | Высокое |

**Итог:** Volume чаще используют для постоянных данных, Bind Mount — для удобной разработки и доступа к файлам хоста.

# Когда что выбирать

### Docker Volume

- Данные PostgreSQL и других СУБД.
- Состояние приложения, независимое от структуры каталогов хоста.
- Управление средствами docker volume.

### Bind Mount

- Исходный код при разработке.
- Конфигурационные файлы и тестовые HTML-страницы.
- Прямой обмен файлами с конкретным каталогом Debian.

# Создаём каталог на хосте

```bash
mkdir -p "$HOME/docker-lab5/site"
printf "Docker bind mount works\n" > \
  "$HOME/docker-lab5/site/index.html"
```

### Что важно

- Каталог создаётся в домашней директории текущего пользователя Debian.
- Файл index.html содержит строку для проверки.
- На хосте можно менять файл обычным редактором.

# Монтируем каталог в Nginx

```bash
sudo docker run -d --name lab5-web \
  -p 8085:80 \
  --mount "type=bind,src=$HOME/docker-lab5/site,dst=/usr/share/nginx/html,readonly" \
  nginx:stable
```

**Что важно**

- type=bind использует каталог Debian.
- readonly защищает подключённый каталог от записи из контейнера.
- Содержимое каталога хоста закрывает файлы, ранее находившиеся по целевому пути контейнера.

# Проверяем bind mount

```bash
curl http://localhost:8085
sudo docker inspect lab5-web
```

**Что важно**

- В ответе ожидается Docker bind mount works.
- В docker inspect найдите Mounts и Type: bind.
- При изменении index.html на хосте ответ Nginx изменится без пересборки образа.

# Осторожно с Bind Mount

- По умолчанию bind mount может предоставлять запись в файлы хоста.
- readonly снижает риск случайного изменения файла со стороны контейнера.
- Если путь источника отсутствует, поведение --mount и -v различается.
- Не монтируйте без необходимости чувствительные каталоги хоста.

# PostgreSQL с постоянным хранилищем

Изучаем постоянное хранение данных без веб-фреймворка и внешних клиентов

# Зачем PostgreSQL отдельный контейнер

- PostgreSQL нужен здесь как наглядный пример приложения, которое постоянно записывает данные на диск.
- Контейнер с PostgreSQL можно заменить, сохранив данные в отдельном Docker Volume.
- Для демонстрации достаточно простых команд psql: CREATE TABLE, INSERT и SELECT.
- Express.js, Node.js и отдельный клиент PostgreSQL не требуются: psql запускается через docker exec.

# Выбор версии PostgreSQL и каталога данных

- Для учебной работы используем официальный Docker-образ postgres:18.
- В postgres:18 значение PGDATA по умолчанию: /var/lib/postgresql/18/docker.
- Именованный том подключаем к /var/lib/postgresql (родительскому каталогу PGDATA).
- В postgres:17 и ранее том обычно подключали к /var/lib/postgresql/data.
- Для PostgreSQL 18 создаём отдельный пустой том: не используем без миграции файлы PostgreSQL 17.

**Главная идея**

**postgres:18 + --mount type=volume,src=pgdata-course18,dst=/var/lib/postgresql**

# Контейнер PostgreSQL и постоянный Docker Volume

Данные базы хранятся в Docker Volume и сохраняются при пересоздании контейнера

## 1 Запуск контейнера PostgreSQL с томом

```
>_ docker volume create pgdata-course

   docker run -d \
     --name pg-course \
     -e POSTGRES_PASSWORD=secret \
     -p 5432:5432 \
     --mount source=pgdata-course,\
       target=/var/lib/postgresql/data \
     postgres:17
```

## 2 Структура




## 3 Что хранится в томе

- pgdata-course
  - base
  - global
  - pg_wal
  - pg_stat
  - pg_tblspc
  - pg_commit_ts
  - pg_dynshmem
  - ...

## 4 Работа с базой данных

```
>_ docker exec -it pg-course psql -U postgres

postgres=# CREATE TABLE categories (
     id SERIAL PRIMARY KEY,
     name  VARCHAR(100)  NOT NULL
     );
INSERT INTO categories (name) VALUES
  ('Электроника'),
  ('Книги');
SELECT * FROM categories;
```

| id | name |
|---|---|
| 1 | Электроника |
| 2 | Книги |

## 5 Пересоздание контейнера без потери данных

**Удаляем контейнер**
(том остается)

```
>_ docker rm -f pg-course
```

**Создаем контейнер заново**
с тем же томом

```
>_ docker run -d --name pg-course ...
```

**Данные остались**
Таблица categories и записи
доступны после пересоздания контейнера

## 6 Итог

Docker Volume хранит данные независимо от жизненного цикла контейнера.

### Важно

- При удалении контейнера том не удаляется.
- Том можно использовать с новым контейнером.
- Для резервного копирования тома используйте `docker run --rm -v ...`

# Создаём том PostgreSQL

BASH

```
sudo docker volume create pgdata-course18
sudo docker volume inspect pgdata-course18
```

**Что важно**

- Создаём новый именованный том pgdata-course18, не используя том со старыми данными PostgreSQL 17.
- Том существует независимо от контейнера и предназначен для каталога /var/lib/postgresql.
- В docker volume inspect проверьте Name и Mountpoint.

# Запускаем PostgreSQL 18

```bash
sudo docker run -d --name pg-course \
  --mount type=volume,src=pgdata-course18,dst=/var/lib/postgresql \
  -e POSTGRES_USER=student \
  -e POSTGRES_PASSWORD=Lab5-ChangeMe-2026 \
  -e POSTGRES_DB=shop \
  postgres:18
```

**Что важно**

- В postgres:18 монтируем том к /var/lib/postgresql, а не /var/lib/postgresql/data.
- POSTGRES_USER, POSTGRES_PASSWORD, POSTGRES_DB инициализируют новый пустой каталог данных.
- Это учебный пароль; не применяйте такую строку запуска с секретом в продуктивной среде.
- Порт 5432 наружу не публикуем. Для работы с базой используем docker exec.

# Проверяем PostgreSQL 18 и PGDATA

BASH

```
sudo docker logs pg-course
sudo docker exec pg-course pg_isready -U student -d shop
sudo docker exec pg-course printenv PGDATA
sudo docker exec pg-course psql -U student -d shop -c "SHOW server_version;"
```

**Что важно**

- После завершения инициализации pg_isready должен сообщить accepting connections.
- printenv PGDATA должен вывести /var/lib/postgresql/18/docker.
- SHOW server_version подтверждает основную версию 18.
- Если контейнер ещё запускается, повторите проверку готовности позже.

# Создаём таблицу через psql

BASH

```
sudo docker exec pg-course \
  psql -U student -d shop \
  -c "CREATE TABLE categories (
    id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name TEXT NOT NULL
  );"
```

**Что важно**

- docker exec запускает psql внутри работающего контейнера.
- Таблица categories создаётся в базе shop.
- Для первого запуска ожидается CREATE TABLE.

# Добавляем учебные записи

BASH

```
sudo docker exec pg-course \
  psql -U student -d shop \
  -c "INSERT INTO categories (name)
      VALUES ('Electronics'),
             ('Books');"
```

**Что важно**

- Команда добавляет две записи.
- В выводе ожидается INSERT 0 2.
- Данные хранятся в томе pgdata-course.

# Запрашиваем данные

```bash
sudo docker exec pg-course \
  psql -U student -d shop \
  -c "SELECT id, name
      FROM categories
      ORDER BY id;"
```

**Что важно**

- Ожидаемые категории: Electronics и Books.
- Сохраните вывод SELECT до пересоздания контейнера.
- Номера id назначаются автоматически.

# Сохранность данных, диагностика и ограничения

Пересоздание контейнера и повторное использование того же тома

# Эксперимент с пересозданием PostgreSQL

| | | |
|---|---|---|
| 1 | **Шаг 1** | Выполнить SELECT и сохранить две категории до пересоздания. |
| 2 | **Шаг 2** | Остановить pg-course командой docker stop. |
| 3 | **Шаг 3** | Удалить только контейнер docker rm; том pgdata-course18 не трогать. |
| 4 | **Шаг 4** | Создать новый контейнер pg-course из postgres:18 с тем же томом и прежним путём /var/lib/postgresql. |
| 5 | **Шаг 5** | Проверить pg_isready, PGDATA и повторить SELECT. |
| 6 | **Шаг 6** | Сравнить записи до и после пересоздания; убедиться, что данные остались. |

# Удаляем только контейнер

```bash
sudo docker stop pg-course
sudo docker rm pg-course
sudo docker volume ls
```

**Что важно**

- Удаляем только контейнер pg-course, не именованный том.
- Том pgdata-course18 по-прежнему должен быть в docker volume ls.
- Не выполняйте docker volume rm, docker volume prune или docker system prune.

# Создаём PostgreSQL повторно с тем же томом

BASH

```
sudo docker run -d --name pg-course \
  --mount type=volume,src=pgdata-course18,dst=/var/lib/postgresql \
  -e POSTGRES_USER=student \
  -e POSTGRES_PASSWORD=Lab5-ChangeMe-2026 \
  -e POSTGRES_DB=shop \
  postgres:18
```

**Что важно**

- Повторно используем тот же том pgdata-course18 и каталог монтирования /var/lib/postgresql.
- Основная версия остаётся 18: существующие файлы PostgreSQL 17 напрямую не подходят.
- При существующем кластере переменные POSTGRES_* не создают базу заново и не меняют пароль пользователя.
- После запуска проверьте готовность PostgreSQL.

# Проверка после пересоздания

```bash
sudo docker exec pg-course pg_isready -U student -d shop
sudo docker exec pg-course \
  psql -U student -d shop \
  -c "SELECT id, name
      FROM categories
      ORDER BY id;"
```

**Что важно**

- В таблице categories должны остаться Electronics и Books с прежними id.
- Данные сохранились в томе pgdata-course18, хотя контейнер был удалён.
- Успешный эксперимент не заменяет резервную копию и проверку восстановления.

# Пересоздание контейнера без потери данных

Как удалить и заново создать Docker-контейнер, сохранив данные в Docker Volume

## 1 Контейнер работает




Таблица categories уже содержит данные.

## 2 Остановка контейнера




```
>_ docker stop
   pg-course
```

Контейнер остановлен. Данные остаются в томе.

## 3 Удаление контейнера




```
>_ docker rm
   pg-course
```

**Контейнер удалён.**

## 4 Docker Volume остаётся




Том НЕ удалён. Данные сохранены на диске Docker.

## 5 Создание нового контейнера




```
>_ docker run -d --name pg-course \
   -e POSTGRES_PASSWORD=secret \
   -v pgdata-course:/var/lib/
   postgresql/data \
   postgres:17
```

Создан новый контейнер, подключён к тому же volume с данными.

## 6 Проверка данных

```
>_ docker exec -it
   pg-course psql -U postgres
```

```
postgres=# SELECT * FROM
categories;
```

| id | name |
|---|---|
| 1 | Электроника |
| 2 | Книги |

**Данные сохранились.**

## Почему данные не потерялись?

- Данные хранятся не в writable layer контейнера.
- Контейнер использует внешний Docker Volume.
- `docker rm` удаляет контейнер, но не том.
- Новый контейнер можно подключить к тому же volume.

## Docker Volume (постоянное хранилище)




Данные хранятся вне контейнера и сохраняются при удалении и пересоздании контейнера.

## Что важно помнить

- **Без volume** данные обычно теряются после удаления контейнера.
- Удаление тома выполняется отдельно (например, `docker volume rm`).
- Volume подходит для БД и постоянных данных.
- Для резервного копирования нужен backup, не только volume.

**Главная идея:** контейнер можно пересоздать, а Docker Volume продолжает хранить данные.

# Типовые ошибки при работе с томами

- Для postgres:18 том по ошибке подключён к /var/lib/postgresql/data вместо /var/lib/postgresql.
- Вместо прежнего тома указано новое имя — база инициализируется пустой.
- Данные из postgres:17 нельзя использовать в postgres:18 без предусмотренной процедуры обновления.
- Изменение POSTGRES_PASSWORD в docker run не меняет пароль уже инициализированной базы.
- Том занят другим контейнером или на диске хоста закончилось место.

# Не путайте volume с резервной копией

- Том обеспечивает постоянство хранения при пересоздании контейнера.
- Удаление тома приводит к утрате его данных, если нет копии.
- Удаление таблицы через SQL тоже изменяет данные внутри тома.
- Резервные копии и восстановление — отдельная задача администратора.

# Как проверить хранилище

| Вопрос | Команда |
| --- | --- |
| Существует ли том? | docker volume ls |
| Куда подключён том? | docker inspect pg-course |
| Где хранится том Docker? | docker volume inspect pgdata-course18 |
| Какой PGDATA в контейнере? | docker exec pg-course printenv PGDATA |
| Работает ли PostgreSQL? | docker exec pg-course pg_isready -U student -d shop |
| Остались ли записи? | docker exec pg-course psql -U student -d shop -c "SELECT COUNT(*) FROM categories;" |

# Итоги лекции

1. При удалении контейнера исчезает его записываемый слой; остановка сама по себе данные не удаляет.
2. Docker Volume хранит данные отдельно от конкретного контейнера.
3. Bind Mount связывает каталог Debian с каталогом контейнера; полезен для файлов сайта.
4. Для официального образа postgres:18 том подключается к /var/lib/postgresql.
5. По умолчанию PGDATA в postgres:18 — /var/lib/postgresql/18/docker.
6. Для повторного запуска используется тот же том и та же основная версия PostgreSQL.
7. Том не является резервной копией.

# Контрольные вопросы

1. 1. Где находятся изменения, если контейнер создан без Volume?
2. 2. Чем отличаются docker stop и docker rm по отношению к записываемому слою?
3. 3. В чём разница между Docker Volume и Bind Mount?
4. 4. Почему для PostgreSQL удобно использовать именованный том?
5. 5. Какой путь монтирования рекомендует официальный образ postgres:18?
6. 6. Какой каталог данных выводит PGDATA в postgres:18?
7. 7. Что произойдёт, если при пересоздании указать другой пустой том?
8. 8. Почему нельзя просто заменить postgres:17 на postgres:18 с прежним томом?
9. 9. Почему после пересоздания должны остаться прежние SQL-записи?
10. 10. Почему Volume нельзя считать резервной копией?